Практическая работа

Отчёт по практической работе № 8

Реализация ролевого управления доступом

Дисциплина:	Основы разработки защищённого программного обеспечения и компьютерных сетей
Вариант:	19 — Система учёта транспортных средств организации
Исходная версия:	v0.7
Версия работы:	v0.8
Студент:	Мостовая Полина Андреевна
Группа:	ИУСбд-02-23

2026

Содержание

1 Тема, цель и среда 2

2 Соответствие v0.5–v0.7 2

3 Модель и матрица доступа 2

4 Интеграция с аутентификацией и SQLite 3

5 Транзакционная последовательность 4

6 Вариант 19: specialist и область Б 4

7 Ключевые фрагменты кода 4

8 Базовые тестовые сценарии 5

9 Дополнительные сценарии варианта 19 5

10 Интеграционные и регрессионные результаты 6

11 Обнаруженные ограничения 6

12 Остаточные риски 7

13 Ручные скриншоты 7

14 Вывод 9

15 Ответы на контрольные вопросы 9

16 Список источников 10

1. Тема, цель и среда

Тема — реализация ролевого управления доступом (RBAC) в сквозном проекте учёта транспортных средств. Цель — связать подтверждённый сеанс v0.7 с текущей серверной ролью, проверить разрешение и принадлежность заявки, а при запрете сохранить защищённые данные неизменными. Дополнительный вариант 19 — повторное открытие закрытой заявки в области Б, то есть только в подразделении specialist.

Работа продолжает локальный проект v0.7. В v0.5 реализованы разбор JSON и `ValidatedRequest`; в v0.6 — `RequestRepository`, SQLite и `FileStore`; в v0.7 — `AuthService`, парольные проверочные значения и серверные сеансы. Существующая модель предметных операций автопарка (`driver`, `dispatcher`, `manager`, `security_admin`) сохранена отдельно. Роли `user`, `operator`, `admin`, `specialist` применяются только к учебным заявкам.

Методичка №8 содержит 14 страниц; требования адаптированы к фактическим модулям. В репозитории нет сохранённой рабочей SQLite-БД. Текущая среда разработки: macOS, Python 3.12.14 в `.venv`, SQLite стандартной библиотеки. Проект не содержит сетевого API.

2. Соответствие v0.5–v0.7

Предыдущая работа	Имеющийся компонент	Использование в v0.8
v0.5	`validate_request`	Контракт содержимого заявки передаётся в защищённый сервис; `owner_id`, роль, active и department не входят в клиентские поля.
v0.6	`RequestRepository`, таблица заявок, `FileStore`	Существующая таблица расширяется без удаления полей; сохраняются категория, приоритет, описание и ключи вложений.
v0.7	`AuthService.resolve`	Адаптер возвращает только серверный ID пользователя для действующего сеанса; парольная проверка не дублируется.
v0.4	`Principal`, роли автопарка	Старая in-memory RBAC-модель остаётся отдельной; названия ролей и политика не меняются.

3. Модель и матрица доступа

RBAC связывает роль с разрешёнными операциями; он не освобождает от проверки конкретного объекта [2, 5]. В этой учебной модели у пользователя одна текущая роль без наследования. Для `user` разрешённое `request.read` дополняется условием `requests.owner_id = users.id`. Для неизвестной роли и операции применяется отказ по умолчанию. Сервер не принимает идентификатор субъекта, роль или атрибут подразделения из заявки [1, 3].

Операция	user	operator	admin	specialist
Создать заявку	Да	Нет	Нет	Нет
Читать заявку	Только свою	Все учебного контура	Нет	Нет
Закрыть `new → closed`	Нет	Да	Нет	Нет
Назначить обычную роль	Нет	Нет	Другому обычному пользователю	Нет
Повторно открыть `closed → new`	Нет	Нет	Нет	Только своё подразделение
Назначить/изменить admin	Нет	Нет	Нет	Нет
Неизвестное действие	Запрет	Запрет	Запрет	Запрет

admin управляет только ролями и не получает права чтения заявок. specialist не получает разрешений operator или admin. Открытая регистрация назначает `user`; первичная административная запись — доверенная локальная подготовка, вне публичной регистрации.

4. Интеграция с аутентификацией и SQLite

AccessService вызывает `AuthService.resolve` и получает только серверный ID для действующего сеанса. Клиент не может передать ID вместо токена. При каждом вызове в общей SQLite-БД читаются текущие `active`, `role` и `department_id`. Роль не встраивается в сессию, поэтому назначение или отзыв применяется при следующем запросе с прежним действующим токеном. Отсутствующая, ошибочная, отозванная или истёкшая сессия и отключённый аккаунт закрывают доступ.

Миграция `AccessService.initialize(path)` повторно запускает инициализацию пользователей и заявок в одном файле. В v0.7 таблица `users` получает отсутствующие `role` (default `user`) и `department_id`; таблица `requests` получает `owner_id`, `status`, `department_id`. Для прежних заявок без владельца/отдела атрибуты остаются `NULL`, состояние — `legacy`. Старые парольные хеши, соли, rounds, recovery-хеши, сеансы и прежние поля заявок не пересоздаются. Миграционный тест проверяет совпадение синтетических данных и идемпотентность.

В checkout нет пользовательских файлов SQLite, которые требовали бы объединения. Сервисы в интегрированном контуре должны получить один путь. Если фактические заполненные файлы существуют отдельно, их автоматическое слияние не реализовано: нужна резервная копия и отдельная контролируемая миграция, до неё нельзя заявлять единый атомарный контур.

5. Транзакционная последовательность

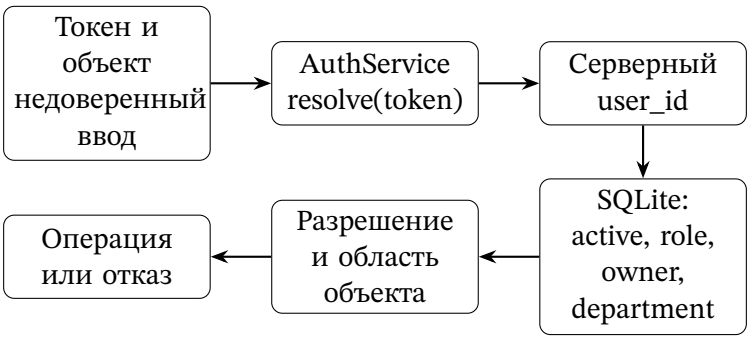


Рис. 1: Подтверждение сеанса и проверка роли/объекта; миграция схемы показана в `docs/authorization_flow.mmd`.

`AuthService.resolve` выполняется до `BEGIN IMMEDIATE`, так как он обновляет `sessions.last_seen`. После этого сервис держит одну транзакцию общей БД на чтение роли, проверку объекта и модификацию заявки. Значения SQL передаются параметрами. При исключении выполняется `ROLLBACK`. Условный `UPDATE` разрешает только `new → closed` или `closed → new` с совпадающим отделом. `BEGIN IMMEDIATE` сериализует пишущие транзакции и снижает параллелизм SQLite [4]. Отзыв полномочия гарантируется для следующего запроса, но не прерывает уже начатую транзакцию.

6. Вариант 19: specialist и область Б

Правило: specialist повторно открывает только заявку со статусом `closed`, если её серверный непустой `department_id` равен текущему серверному отделу пользователя. SQL меняет лишь статус на `new`. NULL, пустое значение, неизвестная заявка, иной статус и другой отдел запрещают действие. Условие не превращает модель в общий ABAC; это дополнительная проверка области объекта поверх одного разрешения `request.reopen`.

Схема варианта 19 показывает этот исполняемый порядок проверок: токен, AuthService, транзакция, текущая роль и отдел, заявка и статус, равенство отделов, затем изменение или отказ. Клиентское поле `department_id` не читается.

7. Ключевые фрагменты кода

Таблица разрешений фиксирована и по умолчанию запрещает неизвестные комбинации:

```
PERMISSIONS = {
    "user": frozenset({"request.create", "request.read"}),
    "operator": frozenset({"request.read", "request.close"}),
    "admin": frozenset({"role.assign"}),
    "specialist": frozenset({"request.reopen"}),
}
```

Переоткрытие проверяет атрибуты из доверенных строк и изменяет только подходящий закрытый объект:

```
cursor = db.execute(
    "UPDATE requests SET status = 'new' "
    "WHERE id = ? AND status = 'closed' AND department_id = ?",
    (request_id, department_id),
)
if cursor.rowcount != 1:
    raise AccessDenied("Операция недоступна")
```

Полная реализация расположена в `app/authorization.py`; схема роли/состояния добавлена к прежним таблицам в `app/auth.py` и `app/repositories/requests.py`.

8. Базовые тестовые сценарии

№	Проверка	Ожидаемый результат	Факт
1	Владелец читает собственную заявку	Заявка доступна	Пройдено
2	Владелец запрашивает чужую заявку	Единый отказ, без раскрытия	Пройдено
3	Operator читает контур	Заявка доступна	Пройдено
4	Operator закрывает новую заявку	`new → closed`	Пройдено
5	User пытается закрыть заявку	Отказ, состояние прежнее	Пройдено
6	Создание заявки владельцем	owner взят из сеанса	Пройдено
7	Подмена `owner_id` в интерфейсе	Аргумент не поддержан	Пройдено
8	Admin читает заявку	Отказ	Пройдено
9	Admin назначает operator	Новое право доступно	Пройдено
10	User назначает роль	Отказ без изменений	Пройдено
11	Operator назначает роль	Отказ без изменений	Пройдено
12	Попытка назначить admin	Отказ без изменений	Пройдено
13	Изменение существующей admin-записи	Отказ без изменений	Пройдено
14	Роль оператора отозвана при действующем сеансе	Следующая операция запрещена	Пройдено
15	Неизвестный/отозванный сеанс	Отказ	Пройдено
16	Учётная запись отключена	Отказ	Пройдено
17	Неизвестная роль	Отказ	Пройдено
18	Неизвестное разрешение	Отказ	Пройдено
19	Запрошена отсутствующая заявка	Единый отказ	Пройдено
20	Повторное закрытие	Отказ, запись не меняется	Пройдено

Базовые проверки находятся в `tests/test_authorization.py`; каждый отказ сравнивает состояние пользователей и заявок до/после.

9. Дополнительные сценарии варианта 19

№	Проверка	Ожидаемый результат	Факт
1	Specialist A открывает закрытую заявку A	Успешно, только статус меняется	Пройдено
2	Specialist A открывает заявку B	Отказ, запись сохранена	Пройдено
3	У пользователя или заявки нет отдела	Отказ, запись сохранена	Пройдено
4	Роль specialist отозвана без нового входа	Сеанс действителен, операция запрещена	Пройдено
5	Заявка уже имеет `new`	Отказ без изменений	Пройдено
6	Operator/Admin пытается выполнить reopen	Отказ без изменений	Пройдено
7	Клиент передаёт отдел A для заявки B	Аргумент не поддержан	Пройдено
8	Заявки не существует	Отказ без изменений	Пройдено

10. Интеграционные и регрессионные результаты

Интеграционные проверки создают пользователей через `AuthService` и выполняют парольный вход. Сеансы передаются `AccessService`; проверяются доступ к своей и чужой заявке, выдача и отзыв operator с прежним сеансом, отключённая запись, выход и истечение по управляемым часам.

Миграционный тест использует искусственные старые таблицы `users`, `sessions` и `requests`; сравнивает соль, хеши, стоимость KDF, recovery-хеш, сеанс и поля заявки.

В отдельном целевом прогоне три файла — `tests/test_authorization.py`, `tests/test_variant19.py` и `tests/test_authorization_integration.py` — прошли 32 теста: 20 базовых, 8 дополнительных и 4 интеграционные проверки. Отдельный интеграционный файл: 4 пройдены. Полный локальный набор: 181 тест и 9 subtests пройдены. Сквозная демонстрация `demo_v08.py--all` завершилась успешно; значения токенов и паролей не выводились. Результат относится к первому проходу и будет повторён перед релизом после добавления снимков.

11. Обнаруженные ограничения

В реальном checkout нет файла данных для миграции, поэтому проверялись синтетические прежние схемы. Если у пользователя есть раздельные непустые БД, их содержимое не смешивается автоматически; требуется резервная копия и контролируемый импорт. В `main.py` нет API/маршрутов заявок, поэтому защита проверена на сервисном интерфейсе и сквозной демонстрации, а не на сетевой точке входа. Это существенно ограничивает утверждение об интеграции с будущим приложением.

Файловая система `FileStore` не участвует в транзакции SQLite; при ошибке после записи применяется компенсирующее удаление. Транзакционный тест относится к пользователям и заявкам в общем файле. `BEGIN IMMEDIATE` блокирует конкурирующие записи; отзыв роли не останавливает текущую операцию. Целостность SQLite зависит от доверия к процессу и ОС. Нет полноценного жизненного цикла администраторов, согласования изменений ролей, многофакторной защиты, распределённого хранилища или

нескольких ролей у пользователя. RBAC не следует трактовать как общую ABAC-систему.

12. Остаточные риски

Кража действующего сеансового токена даёт полномочия до отзыва/истечения. Ошибка доверенного admin может назначить чрезмерную роль. Прямое обращение к `RequestRepository` из будущей внешней точки входа обойдёт сервисную политику. Неполные legacy-атрибуты закрывают специалисту доступ, однако ручное заполнение с ошибкой способно расширить область. Пишущие транзакции SQLite сериализуются; параллельная операция уже начавшаяся до смены роли может завершиться. Компрометация процесса/БД не устраняется тестами. Эти риски описаны также в `docs/authorization.md`, `docs/threat_model.md` и `docs/risk_register.md`.

13. Ручные скриншоты

Семь снимков сделаны в Terminal.app и включены в отчёт. На них показаны подтверждение сеанса и текущая роль, чтение собственной заявки и отказ на чужую, функциональные разрешения, отзыв роли при прежнем сеансе, положительные и отрицательные сценарии варианта 19, полный запуск тестов и версия проекта. Вывод команд сверяется с автоматическими проверками; значения токенов и паролей не отображаются.

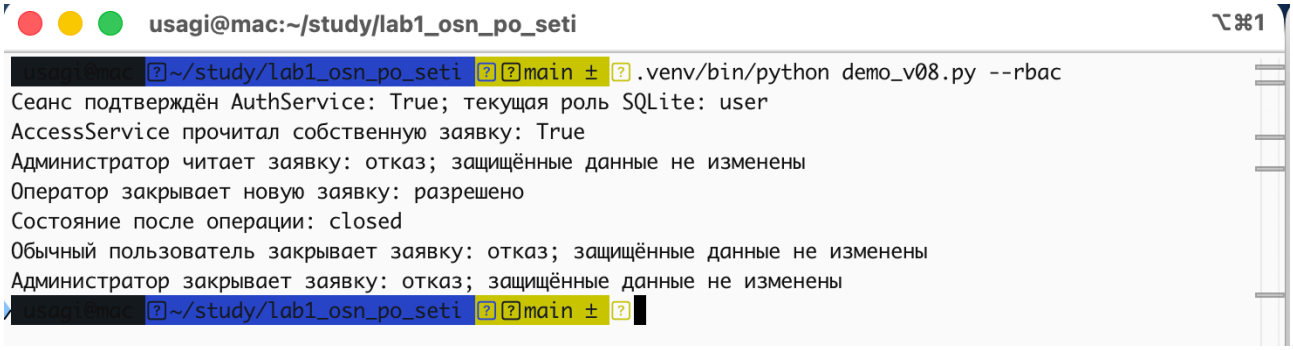


Рис. 2: Подтверждение сеанса AuthService, текущая роль и доступ к собственной заявке.

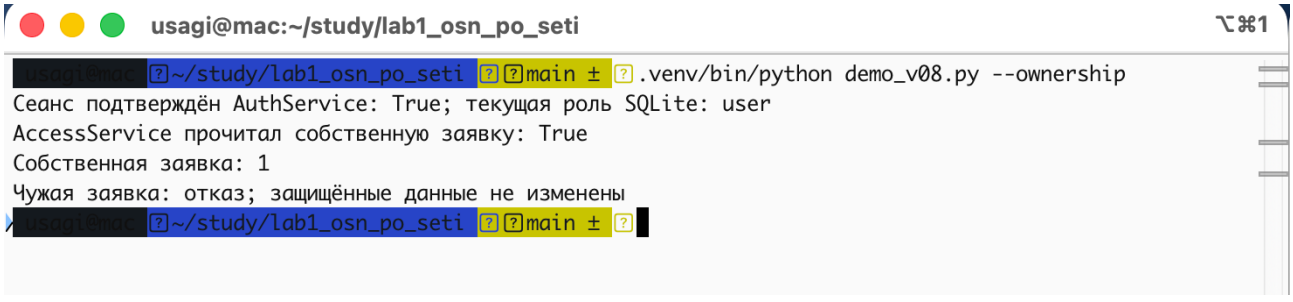


Рис. 3: Объектная авторизация: собственная заявка доступна, чужая отклонена.

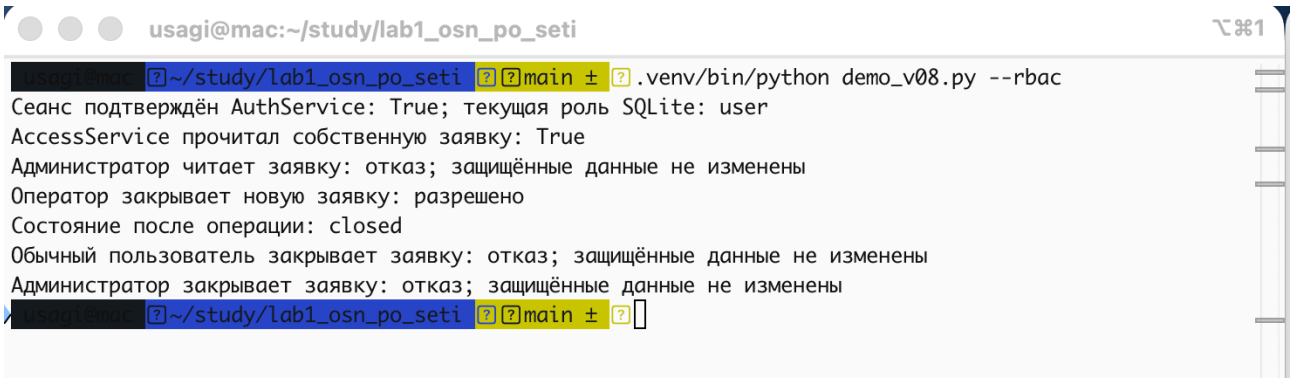


Рис. 4: Функциональные разрешения operator, user и admin.

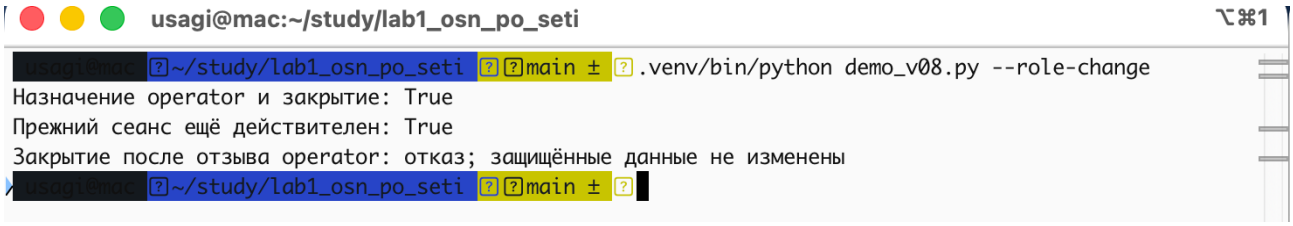


Рис. 5: Отзыв роли operator применяется при следующем обращении с тем же сеансом.

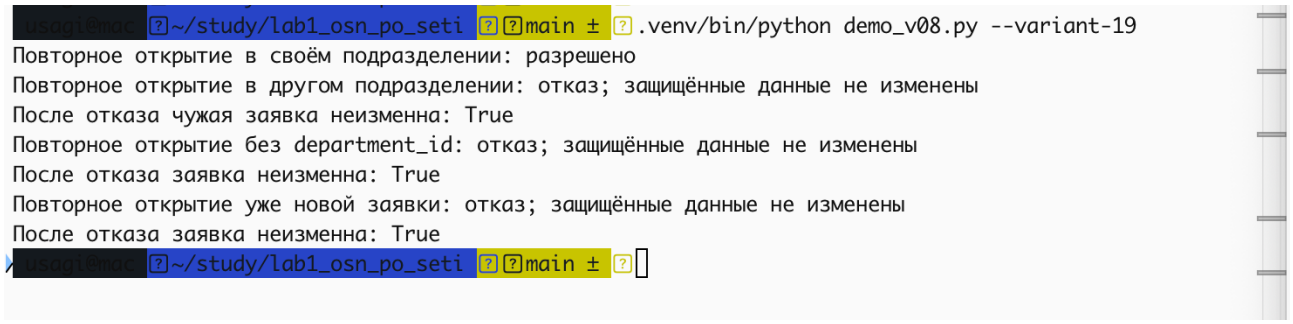


Рис. 6: Вариант 19: специалист повторно открывает заявку своего подразделения.

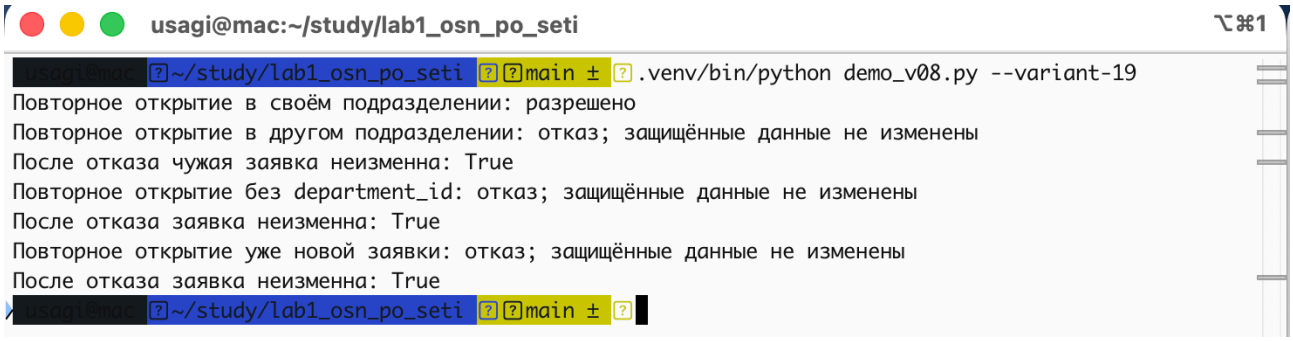


Рис. 7: Вариант 19: чужое подразделение, отсутствующий отдел и недопустимое состояние отклоняются без изменений.

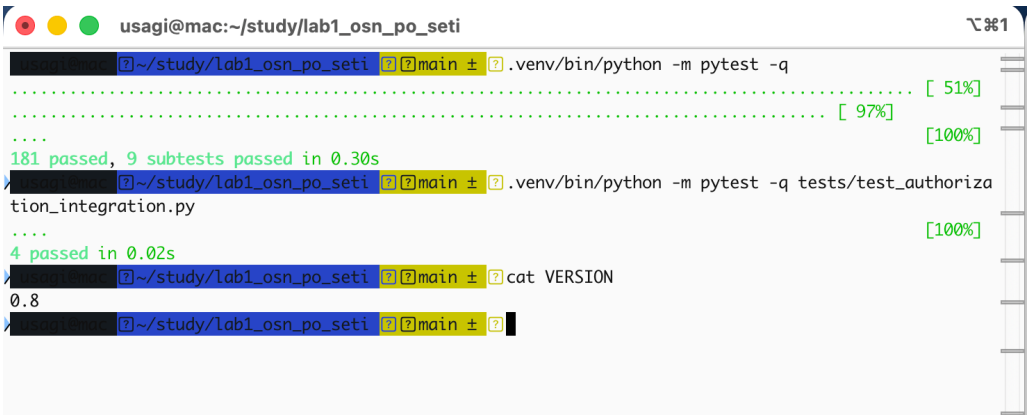


Рис. 8: Полный набор тестов, интеграционные проверки и значение VERSION 0.8.

14. Вывод

В v0.8 добавлен `AccessService`, который подтверждает пользовательский сеанс существующим AuthService, читает актуальные разрешения из общей SQLite-БД и проверяет права на объект. Миграции аддитивны и повторяемы; прежние неизвестные владельцы/отделы не придумываются. Пройдены 20 базовых тестов, восемь дополнительных тестов варианта 19, четыре интеграционные проверки и полный набор из 181 теста с 9 subtests. В отчёт включены семь настоящих терминальных снимков.

15. Ответы на контрольные вопросы

1. Действующий сеанс устанавливает личность, но не решает, имеет ли субъект право на любую конкретную заявку; дополнительно проверяются функция и объект.
2. `request.read` отвечает за доступность функции чтения роли. Проверка владельца сопоставляет подтверждённый ID с владельцем конкретной строки.
3. Клиентский `role` можно подменить; сервер загружает текущую роль из доверенной SQLite-записи.
4. Матрица выдаёт admin только `role.assign`. Принцип минимальных полномочий не даёт ему автоматическое чтение заявок.
5. Сеанс хранит идентичность, а роль повторно загружается при каждом обращении; потому следующий запрос использует уже обновлённую роль.
6. Отказ авторизации означает отсутствие полномочий. Недопустимый переход означает, что право на функцию есть, но текущий объект/статус не допускает переход; оба случая не меняют запись.
7. Параметризация отделяет данные от SQL-синтаксиса, но не проверяет, кому принадлежит строка. Нужна отдельная объектная проверка.
8. Для нескольких ролей потребуются таблицы назначения user-role и role-permission, правила объединения/запрета конфликтующих разрешений, миграция, обновлённая модель сессии и тесты жизненного цикла; их нельзя добавлять простым наследованием в текущую политику.

16. Список источников

1. Методические указания «Практическая работа №8. Реализация ролевого управления доступом». 14 с. Материал, предоставленный для задания.
2. OWASP Foundation. Authorization Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (дата обращения: 29.10.2026).
3. OWASP Foundation. API1:2023 Broken Object Level Authorization. URL: https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/ (дата обращения: 29.10.2026).
4. Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases. URL: https://docs.python.org/3/library/sqlite3.html (дата обращения: 29.10.2026).
5. NIST CSRC. Role-based access control (RBAC), Glossary. URL: https://csrc.nist.gov/glossary/term/role_based_access_control (дата обращения: 29.10.2026).